

Instructions. Show intermediate calculations and explain your reasoning. Work by hand; a calculator may be used to check arithmetic. Exact fractions and logarithms are preferred. No programming is required. Write answers on separate sheets and label each part clearly. All logarithms are natural. Unless stated otherwise, vectors are columns, losses are scalar, and there is no regularization term.

1. Nearest neighbors and the choice of distance

Consider the following training set and a query $q = (0, 0)^\top$:

Training point	Coordinates	Class
$x^{(1)}$	$(3, 0)^\top$	A
$x^{(2)}$	$(2, 2)^\top$	B
$x^{(3)}$	$(0, 4)^\top$	B

- Compute the L^1 and L^2 distances from q to all three training points.
- Predict the class of q using 1-NN with each distance. Explain why the predictions can differ even though the training set is unchanged.
- Predict the class using 3-NN with majority voting. State the prediction for each distance.

2. Linear classifier and softmax loss

A three-class linear classifier has

$$W = (\log 2) \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

The two examples are $x^{(1)} = (1, 0)^\top$ and $x^{(2)} = (0, 1)^\top$, with labels $y^{(1)} = y^{(2)} = 1$. For each example, define

$$z = Wx + b, \quad p_k = \frac{e^{z_k}}{\sum_{j=1}^3 e^{z_j}}, \quad \ell = -\log p_y.$$

- For each example, compute z , all three exponentials, their sum, and p . Check that the probabilities sum to one.
- Compute both individual losses and the mean data loss $L = \frac{1}{2}(\ell^{(1)} + \ell^{(2)})$.
- Replace z by $z + c\mathbf{1}$ for an arbitrary $c \in \mathbb{R}$. Prove that the probabilities and the loss do not change.
- Now consider a C -class softmax classifier whose logits are all equal. Compute each class probability p_k and the cross-entropy loss $\ell = -\log p_y$ in terms of C .

3. What does a regularizer prefer?

For a scalar linear prediction, let

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad w_A = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad w_B = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \hat{y} = w^\top x.$$

The training set consists only of this input with target $y = 2$. Define

$$\ell(w) = \frac{1}{2}(w^\top x - 2)^2, \quad R_1(w) = \lambda \|w\|_1, \quad R_2(w) = \frac{\lambda}{2} \|w\|_2^2, \quad \lambda > 0.$$

- (a) Compute the predictions and data losses for w_A and w_B .
- (b) Compute both penalties for each weight vector. Compare $\ell + R_1$ and $\ell + R_2$: which weight vector is preferred, or are they tied?
- (c) Evaluate the claim: “L1 regularization always assigns a strictly smaller penalty to a vector with more zero entries.” Use your calculations to justify your answer.
- (d) For a differentiable data loss $D(w)$, derive $\nabla_w [D(w) + R_2(w)]$. Rewrite one gradient descent step in the form

$$w_{\text{new}} = \alpha w - \eta \nabla_w D(w),$$

and identify α . Explain the effect of the regularization term when $0 < \eta\lambda < 1$.

4. A gradient describes local change

Let

$$f(x_1, x_2) = \frac{1}{2}x_1^2 + 2x_2^2, \quad a = (2, 1)^\top.$$

- (a) Compute $\nabla f(x)$ and $\nabla f(a)$. Interpret each component of $\nabla f(a)$ as a rate of change.
- (b) For $\Delta x = (0.1, -0.1)^\top$, use

$$f(a + \Delta x) - f(a) \approx \nabla f(a)^\top \Delta x$$

to estimate the change. Compute the actual change and the difference between the two values.

- (c) For a unit vector u , the directional derivative is $D_u f(a) = \nabla f(a)^\top u$. Find the unit direction that makes this value as negative as possible, and justify your answer. Why is the unit-length restriction needed?
- (d) Compute $f(a - \eta \nabla f(a))$ for $\eta = 0.1$ and $\eta = 1$. Does moving in the negative-gradient direction guarantee a smaller function value for every positive step size? Explain using your results.

5. SGD for a linear softmax classifier

Consider a two-class linear classifier with trainable $W \in \mathbb{R}^{2 \times 2}$ and no bias. Its training set is

Example i	Input $x^{(i)}$	Class label $y^{(i)}$
1	$(1, 0)^\top$	1
2	$(0, 1)^\top$	1
3	$(1, 1)^\top$	2

For each example, define

$$z^{(i)} = Wx^{(i)}, \quad p^{(i)} = \text{softmax}(z^{(i)}), \quad \ell_i(W) = -\log p_{y^{(i)}}^{(i)}, \quad F(W) = \frac{1}{3} \sum_{i=1}^3 \ell_i(W).$$

For $z = (z_1, z_2)^\top$, the softmax function is defined by

$$\text{softmax}(z) = \frac{1}{e^{z_1} + e^{z_2}} \begin{pmatrix} e^{z_1} \\ e^{z_2} \end{pmatrix}.$$

You may use the gradient formula

$$G_i(W) := \nabla_W \ell_i(W) = (p^{(i)} - e_{y^{(i)}})(x^{(i)})^\top, \quad e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

An SGD step samples one index i uniformly from $\{1, 2, 3\}$ and updates

$$W_{t+1} = W_t - \eta G_i(W_t).$$

Use $W_0 = 0_{2 \times 2}$ and $\eta = \log 3$.

- Initial values.** For each $i = 1, 2, 3$, compute the probability vector $p^{(i)}$, the loss $\ell_i(W_0)$, and the gradient matrix $G_i(W_0)$ at $W = W_0$.
- Full-batch GD.** Compute both matrices:

$$\bar{G} = \frac{G_1(W_0) + G_2(W_0) + G_3(W_0)}{3}, \quad W_{\text{GD}} = W_0 - \eta \bar{G}.$$

- One SGD step.** Using example 1, compute

$$W_1 = W_0 - \eta G_1(W_0), \quad \ell_1(W_1), \quad F(W_1).$$

State whether $\ell_1(W_1)$ is larger or smaller than $\ell_1(W_0)$, and whether $F(W_1)$ is larger or smaller than $F(W_0)$.

- Order of examples.** Starting from W_0 in each case, compute the two final matrices $W_{1 \rightarrow 3}$ and $W_{3 \rightarrow 1}$ defined by

$$\begin{aligned} W_A &= W_0 - \eta G_1(W_0), & W_{1 \rightarrow 3} &= W_A - \eta G_3(W_A), \\ W_B &= W_0 - \eta G_3(W_0), & W_{3 \rightarrow 1} &= W_B - \eta G_1(W_B). \end{aligned}$$

The first row uses example 1 then example 3; the second uses example 3 then example 1. Explain why $W_{1 \rightarrow 3}$ and $W_{3 \rightarrow 1}$ differ.

6. Adam

Suppose Adam computes the following gradients at its current parameters in steps 1, 2, and 3:

$$g_1 = \begin{pmatrix} 2 \\ 20 \end{pmatrix}, \quad g_2 = \begin{pmatrix} 2 \\ -20 \end{pmatrix}, \quad g_3 = \begin{pmatrix} 2 \\ 20 \end{pmatrix}.$$

Use $\theta_0 = (0, 0)^\top$, $\eta = 0.1$, $\beta_1 = \beta_2 = \frac{1}{2}$, and $T = 3$. For this hand calculation only, set $\epsilon = 0$; every denominator below is nonzero.

Adam algorithm

Input: θ_0 , number of steps T , learning rate η , β_1 , β_2 , ϵ .

- 1 Initialize $m_0 = 0$ and $v_0 = 0$ (same shape as θ_0).
- 2 **For** $t = 1, \dots, T$:
- 3 Obtain the current gradient g_t .
- 4 $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)g_t$
- 5 $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)(g_t \odot g_t)$
- 6

$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$

Bias correction
- 7 $\Delta\theta_t \leftarrow -\eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
- 8 $\theta_t \leftarrow \theta_{t-1} + \Delta\theta_t$
- 9 **End for**
- 10 **Return** θ_T .

Use the given g_t in line 3. All vector operations in the updates are elementwise. Show your calculations.

- (a) **Adam updates.** Compute $m_t, v_t, \hat{m}_t, \hat{v}_t, \Delta\theta_t$, and θ_t for $t = 1, 2, 3$.
- (b) **SGD comparison.** The first gradient is $g_1 = (2, 20)^\top$: its first component is 2 and its second component is 20. Compute $\Delta\theta_1^{\text{SGD}} = -\eta g_1$. For SGD and Adam separately, write the first parameter change as $\Delta\theta_1 = (a, b)^\top$, where a and b are the changes to the first and second parameter coordinates. Compute $|b|/|a|$ for each optimizer. Why do the ratios differ?
- (c) **Gradient signs.** The second component of g_t is 20, $-20, 20$. Using your results from (a), explain why the second component of m_t changes sign but that of v_t does not.
- (d) **Bias correction.** Compute $\Delta\theta_1$ without the bias correction in line 6: use m_1, v_1 instead of $\hat{m}_1, \hat{v}_1$ in line 7. Is each component larger or smaller in magnitude than the corrected $\Delta\theta_1$ from (a)?

7. Chain rule with the course's column convention

Convention used in this assignment. For $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$, define

$$\left(\frac{\partial y}{\partial x}\right)_{ij} = \frac{\partial y_j}{\partial x_i}, \quad \frac{\partial y}{\partial x} \in \mathbb{R}^{n \times m}.$$

Thus, for a scalar loss L ,

$$\underbrace{\nabla_x L}_{n \times 1} = \underbrace{\frac{\partial y}{\partial x}}_{n \times m} \underbrace{\nabla_y L}_{m \times 1}.$$

This fixes the arrangement of Jacobian entries as well as the use of column vectors. State the shape of every derivative in this problem.

- (a) **Scalar to scalar.** Let $u = t^2 + 1$ and $L = u^3$. Compute dL/dt by the chain rule and by first substituting u into L . Evaluate at $t = 1$.
- (b) **Vector to scalar.** Let $x \in \mathbb{R}^2$, $u = x_1 + 2x_2$, and $L = u^2$. Compute $\partial u / \partial x$, dL/du , and $\nabla_x L$. Check by direct differentiation and evaluate at $x = (1, -1)^\top$.
- (c) **Vector to vector.** Define

$$h = f(x) = \begin{pmatrix} x_1 + x_2 \\ x_1 x_2 \\ x_1^2 \end{pmatrix}, \quad y = g(h) = \begin{pmatrix} h_1^2 + h_2 \\ h_3 - h_2 \end{pmatrix}.$$

Compute $\partial h / \partial x$ and $\partial y / \partial h$. Multiply them in the order required by the convention to obtain $\partial y / \partial x$. Verify the result by differentiating y directly as a function of x .

- (d) **Backpropagating a scalar loss.** Use h and y from (c), and let $L = y_1 + 3y_2$. At $x = (1, 2)^\top$:
 - (i) Compute the vectors h, y and the scalar L .
 - (ii) Compute the three column vectors

$$\frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial h} = \frac{\partial y}{\partial h} \frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial x} = \frac{\partial h}{\partial x} \frac{\partial L}{\partial h}.$$

Use the Jacobians from (c), evaluated at the corresponding values of x, h . Show the dimensions of the matrices and vectors in both products.

- (iii) Express L using only x_1, x_2 . Differentiate this expression and evaluate at $(1, 2)^\top$. Check that the result equals $\partial L / \partial x$ from (ii).

8. Why do neural networks need nonlinear activation?

- (a) Consider two affine layers with no activation between them:

$$f(x) = W_2(W_1x + b_1) + b_2.$$

Find W and b such that $f(x) = Wx + b$ for every x .

- (b) Let

$$r(x) = \text{ReLU}(x_1 - x_2) + \text{ReLU}(x_2 - x_1), \quad \text{ReLU}(t) = \max\{0, t\}.$$

Compute $r(x)$ for $x = (0, 0)^\top, (1, 0)^\top, (0, 1)^\top, (1, 1)^\top$. These outputs form the XOR function on binary inputs.

- (c) Can an affine function $w_1x_1 + w_2x_2 + b$ match all four outputs from (b)? Justify your answer by substituting the four inputs.

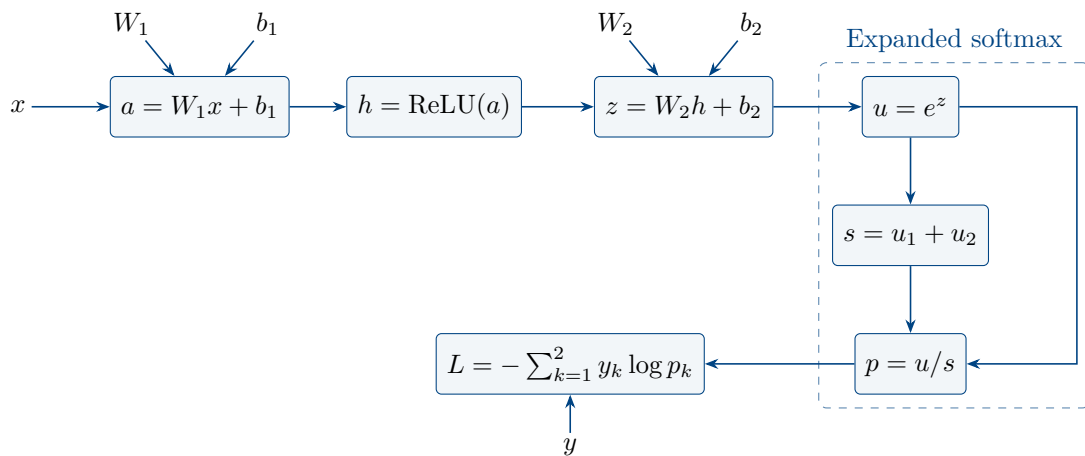
9. Forward and backward in a two-layer network

Use the following input, parameters, and one-hot target:

$$x = \begin{pmatrix} 1 \\ -1 \\ 2 \end{pmatrix}, \quad W_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & -1 \end{pmatrix}, \quad b_1 = 0 \in \mathbb{R}^4,$$

$$W_2 = \begin{pmatrix} 1 & -1 & 0 & 1 \\ -1 & 1 & 1 & -1 \end{pmatrix}, \quad b_2 = 0 \in \mathbb{R}^2, \quad y = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The graph applies to any input and target; use the given values for the calculations. ReLU and the exponential act elementwise. The target y is fixed and is not differentiated.



- (a) **Forward.** Compute a, h, z, u, s, p , and L .
- (b) **Backward.** Starting with $\partial L / \partial L = 1$, compute

$$\frac{\partial L}{\partial p}, \quad \frac{\partial L}{\partial s}, \quad \frac{\partial L}{\partial u}, \quad \frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial h}, \quad \frac{\partial L}{\partial a}, \quad \frac{\partial L}{\partial x},$$

$$\frac{\partial L}{\partial W_1}, \quad \frac{\partial L}{\partial b_1}, \quad \frac{\partial L}{\partial W_2}, \quad \frac{\partial L}{\partial b_2}.$$

- (c) **Zero initialization.** Reset all four parameter arrays to zero and use $\text{ReLU}'(0) = 0$. Compute h, p and all four parameter gradients. Perform one gradient descent update with learning rate $\eta = 1$. Which parameter arrays remain zero under repeated updates?

10. Memory for forward and backward

Use the graph in Problem 9. Count each stored value once, including the input, parameters, and intermediate results. Keep all these values until backward ends. Every real entry uses 4 bytes. Exclude the fixed target y , constants, and temporary workspace.

For the initial backward implementation, also store:

- one gradient array for each stored value, including the scalar $\partial L / \partial L = 1$;
- separate dense arrays for $J_1 = \partial a / \partial x$ and $J_2 = \partial z / \partial h$.

No other Jacobians are stored. Use the column convention from Problem 7.

- (a) **Forward storage.** Count the stored real entries and the required bytes.
- (b) **Backward storage.** Count the additional bytes for the gradient arrays and for J_1, J_2 . Give the total memory with the forward values retained.
- (c) **Reuse the weight matrices.** Show that $J_1 = W_1^\top$ and $J_2 = W_2^\top$. Remove the separate Jacobian copies and use the existing weight arrays for backward. Keeping all other storage rules unchanged, compute the bytes saved and the new total memory.